

S.Y.B.SC.(DATA SCIENCE) SEM- III

Subject: Data Structure – I

Unit2 : Array as a Data Structure

By,

Mrs. Reshma Masurekar

Dr. D. Y. Patil ACS College,

Pimpri, Pune 18.

Searching

- **Searching** is the process of finding a particular element in the list.
- If the element is present in the list, then the process is called successful, and the process returns the location of that element; otherwise, the search is called unsuccessful.
- Two popular search methods are
 - Linear Search
 - Binary Search

Linear Search

- Linear search is also called as **sequential search algorithm**.
- It is the Simplest searching algorithm.
- Traverse the list completely and match each element of the list with the Key whose location is to be found. If the match is found, then the location of the key is returned; otherwise, the algorithm returns NULL.
- It is widely used to search an element from the unordered list.

Working of Linear search

Let's consider an unsorted array. Let the elements of array are -

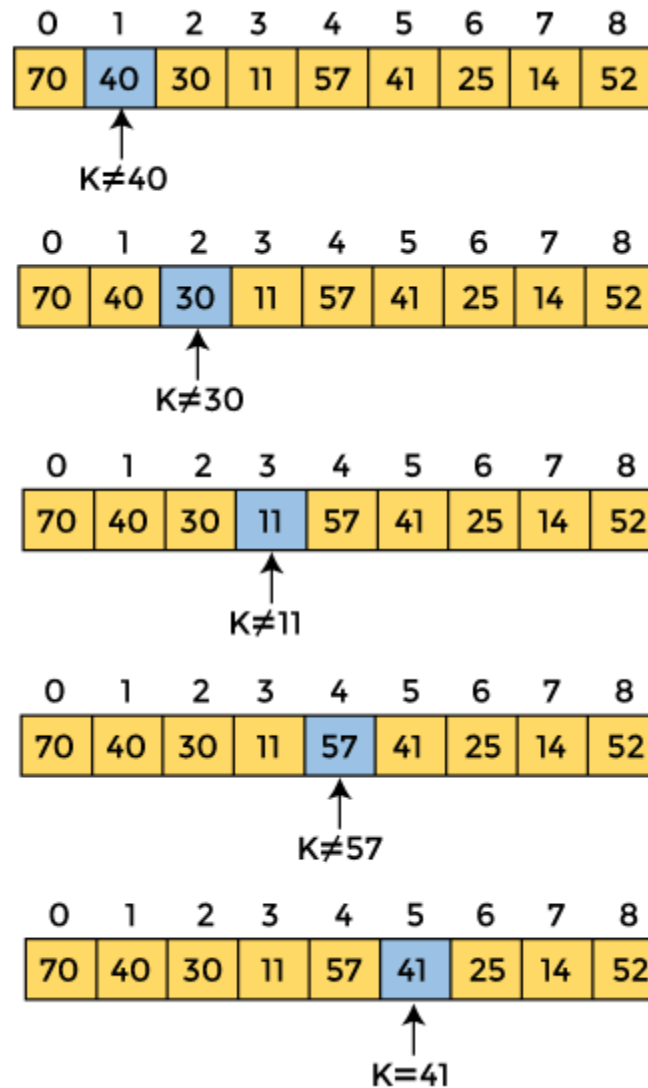
0	1	2	3	4	5	6	7	8
70	40	30	11	57	41	25	14	52

- Let the element to be searched is $K = 41$
- Now, start from the first element and compare K with each element of the array.

0	1	2	3	4	5	6	7	8
70	40	30	11	57	41	25	14	52

↑
 $K \neq 70$

- The value of K , i.e., **41**, is not matched with the first element of the array. So, move to the next element. And follow the same process until the respective element is found.



Now, the element to be searched is found. So algorithm will return the index of the element matched.

Linear Search Time Complexity

- **Best Case:** The best-case time complexity of linear search is **$O(1)$** . Best case occurs when the element we are finding is at the first position of the array.
- **Average Case:** The average case time complexity of linear search is **$O(n)$** .
- **Worst Case:** The worst-case time complexity of linear search is **$O(n)$** . The worst case occurs when the key we have to search is present at the end of the array. The worst-case in linear search could be when the target element is not present in the given array, and we have to traverse the entire array.
- The time complexity of linear search is **$O(n)$** because every element in the array is compared only once.

Binary Search

- A Binary Search is a search algorithm, that is used to search a key in a sorted array.
- Binary Search is defined as a searching algorithm *used in a sorted array by repeatedly dividing the search interval in half*.
- **Prerequisite:** Binary search requires elements to be in sorted order.
- **Working:** Binary search follows the **divide and conquer approach** in which the list is divided into two parts, and the element is compared with the middle element of the list. If the match is found then, the location of the middle element is returned. Otherwise, we search into either of the parts/halves depending upon the result produced through the match.
- Binary search is **better** than the linear search technique as the number of iterations decreases in the binary search.
- **Space complexity is $O(1)$.**

Working of Binary search

- Divide the search space into two halves by finding the mid index value.
- Compare the middle element of the search space with the **key**.
- If the **key** is found at middle element, the process is terminated.
- If the **key** is not found at middle element, then do the following:
 - If the **key** is smaller than the middle element, then the **left part** is used for next search.
 - If the **key** is larger than the middle element, then the **right** part is used for next search.
- This process is continued until the **key** is found or the total search space is exhausted.

Binary search implementation

- Iterative
- Recursive

Binary search

To implement the Binary Search algorithm we need:

1. An array with values to search through.
2. A target value to search for.
3. A loop that runs as long as left index is less than, or equal to, the right index.
4. An if-statement that compares the middle value with the target value, and returns the index if the target value is found.
5. An if-statement that checks if the target value is less than, or larger than, the middle value, and updates the "left" or "right" variables to narrow down the search area.
6. After the loop, return -1, because at this point we know the target value has not been found.

Binary search implementation

Recursive way

```
binarySearch(arr, x, low, high)
    if low > high
        return False
    else
        mid = (low + high) / 2
        if x == arr[mid]
            return mid
        else if x > arr[mid]
            return binarySearch(arr, x, mid + 1, high)
        else
            return binarySearch(arr, x, low, mid - 1)
```

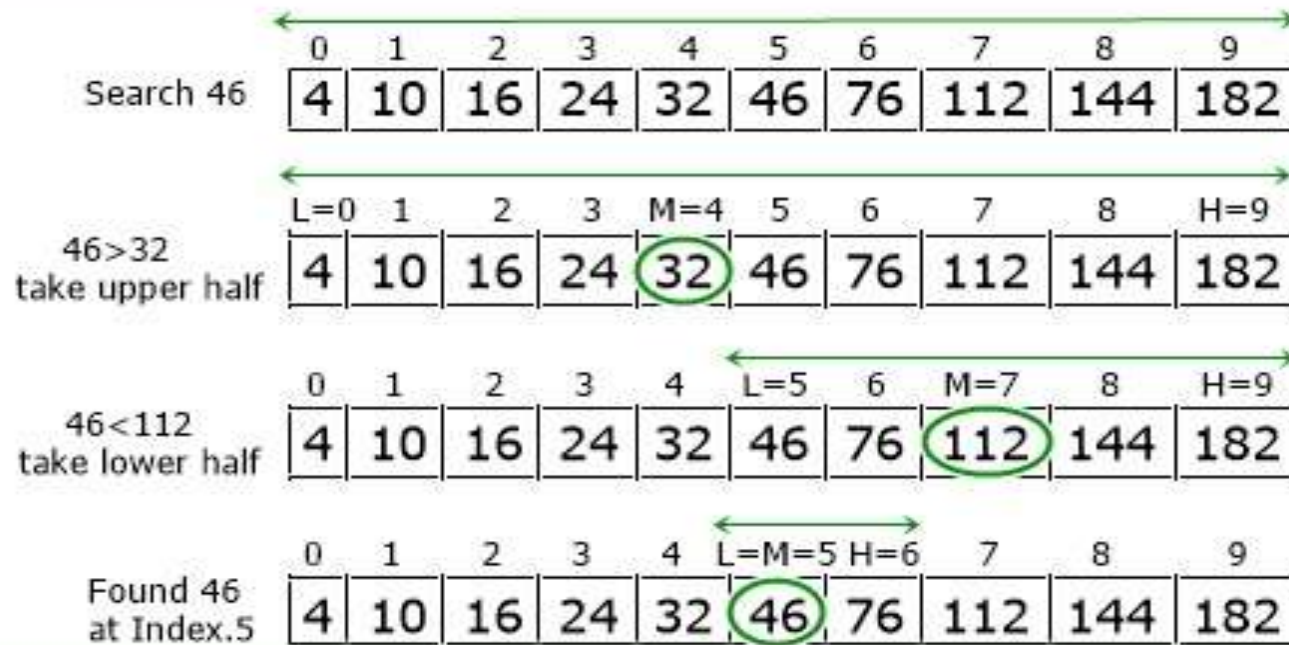
Iterative Way

```
mid = (low + high) / 2
if (x == arr[mid])
    return mid
else if (x > arr[mid])
    low = mid + 1
else
    high = mid - 1
```

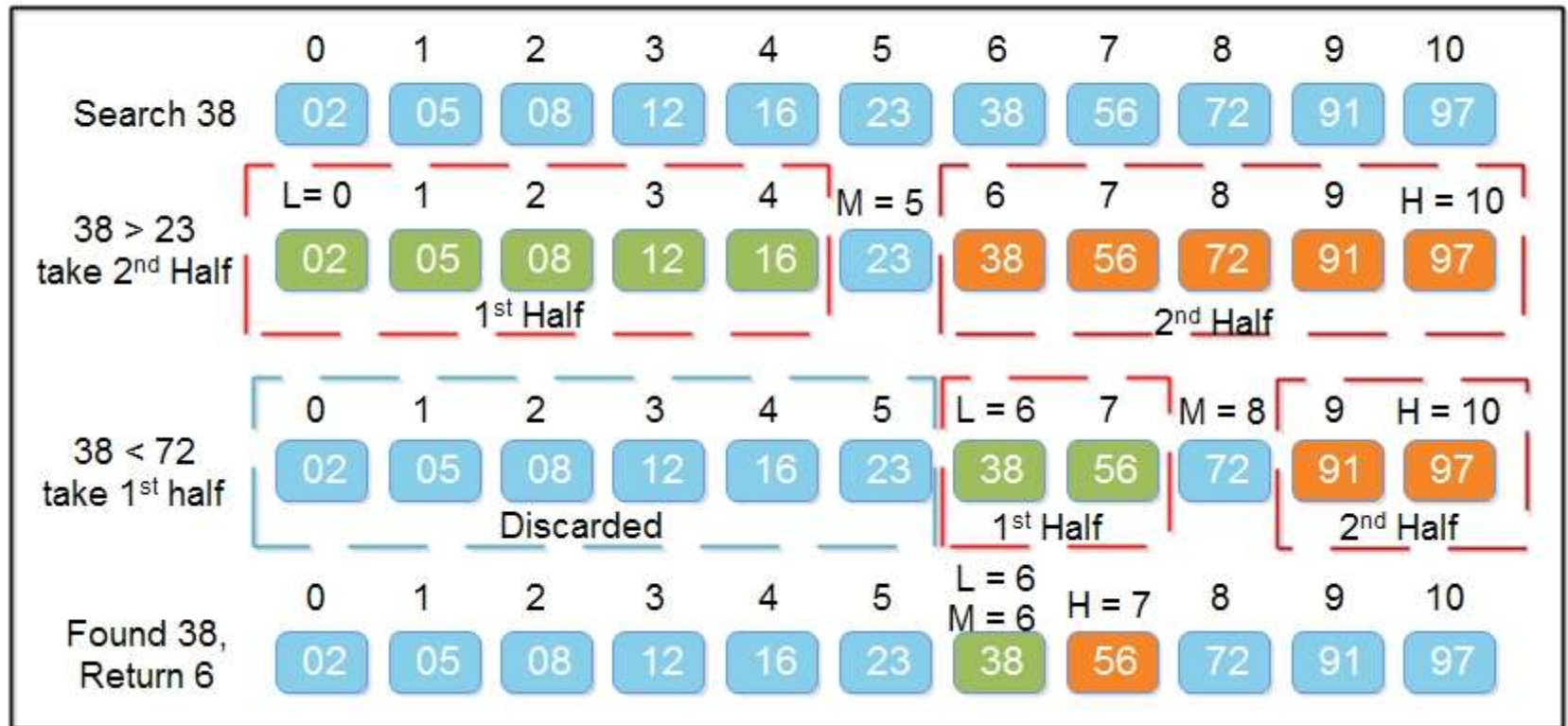
Binary Search

	0	1	2	3	4	5	6	7	8	9
Search 23	2	5	8	12	16	23	38	56	72	91
	L=0				M=4					H=9
23 > 16 take 2 nd half	2	5	8	12	16	23	38	56	72	91
						L=5		M=7		H=9
23 < 56 take 1 st half	2	5	8	12	16	23	38	56	72	91
						L=5, M=5	H=6			
Found 23, Return 5	2	5	8	12	16	23	38	56	72	91

Binary Search



Binary Search in C



Time Complexity and Space Complexity

- **Best Case Complexity** - In Binary search, best case occurs when the element to search is found in first comparison, i.e., when the first middle element itself is the element to be searched. The best-case time complexity of Binary search is **$O(1)$** .
- **Average Case Complexity** - The average case time complexity of Binary search is **$O(\log n)$** .
- **Worst Case Complexity** - The worst case occurs, when we have to keep reducing the search space till it has only one element. The worst-case time complexity of Binary search is **$O(\log n)$** .
- The space complexity of binary search is $O(1)$.

Case	Time Complexity
Best Case	$O(1)$
Average Case	$O(\log n)$
Worst Case	$O(\log n)$

Space Complexity

- The space complexity of binary search is $O(1)$.


```
def binarySearch(arr, targetVal):  
    left = 0  
    right = len(arr) - 1  
  
    while left <= right:  
        mid = (left + right) // 2  
  
        if arr[mid] == targetVal:  
            return mid  
  
        if arr[mid] < targetVal:  
            left = mid + 1  
        else:  
            right = mid - 1  
  
    return -1
```

```
mylist = [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]  
x = int(input("Enter the value to search : "))  
  
result = binarySearch(mylist, x)  
  
if result != -1:  
    print("Found at index", result)  
else:  
    print("Not found")
```

```
def binarySearch(arr, low, high, x):  
    if high >= low:  
        mid = low + (high - low) // 2  
        if arr[mid] == x:  
            return mid  
        elif arr[mid] > x:  
            return binarySearch(arr, low,  
mid-1, x)  
  
        else:  
            return binarySearch(arr, mid +  
1, high, x)  
  
    else:  
        return -1
```

```
arr = [2, 3, 4, 10, 40]  
x = int(input("Enter the value to search : "))  
  
result = binarySearch(arr, 0, len(arr)-1, x)  
  
if result != -1:  
    print("Element is present at index", result)  
else:  
    print("Element is not present in array")
```

- **Advantages:**

- **Efficient:** Binary search has a time complexity of $O(\log n)$, which makes it very efficient for searching large sorted arrays.
- **Simple to implement:** Binary search is relatively easy to implement and understand.
- **Versatile:** Binary search can be used in a wide variety of applications.
- **Reliable:** Binary search is a reliable algorithm that will always find the target element if it exists in the array.

- **Disadvantages:**

- **Requires a sorted array:** Binary search only works on sorted arrays. If the array is not sorted, it must be sorted before binary search can be used.
- **Not suitable for unsorted data:** Binary search is not suitable for searching unsorted data, as it will not be able to find the target element efficiently.
- **May not be the best choice for large arrays:** For very large arrays (e.g., billions of elements), other search algorithms such as interpolation search or hash tables may be more efficient.